JWT ve Refresh Token

Bu projede giriş sistemi nasıl çalışıyor? En baştan, en basit haliyle, gerçek kodlarla.

Bu rehber **wallet** projesinin kendi kodunu anlatır. Hiçbir şey bildiğin varsayılmaz. Her kavram önce günlük hayattan bir benzetmeyle, sonra projedeki gerçek kodla anlatılır.

Etiketler: MOBİL telefondaki uygulama (Expo / React Native) · BACKEND sunucu (Spring Boot) · VERİTABANI PostgreSQL

1. Önce hikâye: otel benzetmesi
2. Temel kavramlar (sıfırdan)
3. Projenin haritası: kim ne yapar?
4. Backend sınıfları tek tek
5. Mobil dosyalar tek tek
6. Senaryo: Giriş yapmak
7. Senaryo: Normal bir istek
8. Senaryo: 15 dakika doldu
9. Senaryo: Aynı anda 3 istek
10. Senaryo: Çıkış yapmak
11. Senaryo: Uygulamayı kapatıp açmak
12. Senaryo: Token çalındı
13. Senaryo: Kullanıcı banlandı
14. Veritabanı tablosunu okumak
15. Hash karşılaştırması: BCrypt ve SHA-256
16. Token çalınırsa: sistemin sınırları
17. Mutlak oturum süresi (henüz yok)
18. Süreler: ne kadar olmalı?
19. Hırsızın elini kolunu bağlamak: güvenlik katmanları
20. Sık sorulan "saf" sorular
21. Tek sayfalık özet ve yapılacaklar

---

1. Önce hikâye: otel benzetmesi

Bütün sistemi tek bir hikâyeyle anlayabilirsin. Bir otele gittiğini düşün.

1. **Resepsiyona gidersin** ve kimliğini gösterirsin. Bu **giriş yapmak (login)**: kullanıcı adı ve şifre.
2. Resepsiyon sana **iki şey** verir:
   - 🔑 **Oda kartı:** Kapıları açar. Ama **15 dakika** sonra kendiliğinden çalışmaz olur. Bu **access token**.
   - 📄 **Konaklama belgesi:** Kapı açmaz. Sadece resepsiyona gösterip **yeni oda kartı** almaya yarar. **30 gün** geçerlidir. Bu **refresh token**.
3. **Her kapıda** (odan, spor salonu, havuz) kartını okutursun. Kapıdaki okuyucu kartın sahte olmadığına ve süresinin dolmadığına bakar. Bu, **her istekte access token'ın kontrol edilmesi**.
4. 15 dakika sonra kart çalışmaz. Kapı açılmaz (**401**). Sen de hiç üşenmeden resepsiyona gidip belgeni gösterirsin ve yeni kart alırsın. Resepsiyon **eski belgeni yırtar** ve sana **yeni bir belge** verir. Bu **refresh**. Bizim uygulamada bunu telefon senin yerine, sen fark etmeden yapıyor.
5. Otelden ayrılırken belgeni resepsiyona verirsin, resepsiyon onu **iptal eder**. Artık o belgeyle kimse yeni kart alamaz. Bu **çıkış yapmak (logout)**.
6. Biri senin **eski, yırtılmış** belgenin fotokopisiyle resepsiyona gelirse resepsiyon şöyle der: "Bu belge zaten kullanılmıştı. Biri çalmış!" Güvenlik için senin **bütün** belgelerini iptal eder. Bu **tekrar kullanım tespiti (reuse detection)**.

Neden iki ayrı şey?

Oda kartı her kapıda kullanıldığı için kaybolma veya kopyalanma ihtimali yüksek. Bu yüzden süresi kısa: çalınsa bile 15 dakika sonra işe yaramaz. Konaklama belgesi ise sadece resepsiyonda kullanılıyor, yani çok daha az ortalıkta dolaşıyor. Bu yüzden süresi uzun tutulabilir. Ve resepsiyon (veritabanı) onu istediği an iptal edebilir.

Otelde	Projede
Resepsiyon	BACKEND `AuthController` + `AuthService`
Kimlik göstermek	`POST /api/auth/login` (kullanıcı adı + şifre)
Oda kartı (15 dk)	**Access token** (JWT), `JwtService` üretir
Konaklama belgesi (30 gün)	**Refresh token**, `RefreshTokenService` üretir
Resepsiyonun belge defteri	VERİTABANI `refresh_tokens` tablosu
Kapıdaki kart okuyucu	`JwtAuthenticationFilter`
Hangi kapı kart ister, hangisi istemez	`SecurityConfig`
Cebindeki cüzdan	MOBİL `tokenStorage` (telefonun şifreli kasası)
Kartı kapıya okutan ve gerekirse resepsiyona koşan sen	MOBİL `apiClient`
Yeni kart almak	`POST /api/auth/refresh`
Belgeyi iade etmek	`POST /api/auth/logout`

2. Temel kavramlar (sıfırdan)

2.1 Kimlik doğrulama (authentication) nedir?

Sunucuya "harcamalarımı getir" dediğinde, sunucunun **senin kim olduğunu** bilmesi gerekir. Yoksa herkes herkesin harcamasını görürdü. Sunucu seni her istekte tanımalı.

Ama sunucu her istekte şifreni soramaz; her ekranda şifre yazmak zorunda kalırdın. Çözüm: Bir kez şifreni söylersin, sunucu sana bir **token** (jeton, giriş kartı) verir. Sonraki isteklerde şifre yerine bu token'ı gösterirsin.

2.2 Token nedir?

Token, "bu kişi giriş yaptı" anlamına gelen bir **yazıdır**. Uzun, anlamsız görünen bir metin. Telefon onu saklar ve her isteğe ekler:

```
GET /api/expenses
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJtdXJhdCIsImV4cCI6...
               └────┘ └──────────────────────── token ──────────────────────────┘
                 │
                 └─ "taşıyıcı": bu token'ı taşıyan kişi içeri alınsın
```

2.3 JWT nedir?

JWT (JSON Web Token), access token'ın biçimidir. Noktayla ayrılmış **üç parçadan** oluşur:

```
eyJhbGciOiJIUzI1NiJ9 . eyJzdWIiOiJtdXJhdCIsImlhdCI6MTc5MzI3MDQwMCwiZXhwIjoxNzkzMjcxMzAwfQ . -MOAfLIHUDupO7lmmnwJf0
└──── BAŞLIK ──────┘   └──────────────────────── İÇERİK (payload) ──────────────────────────┘   └──── İMZA ──────┘

BAŞLIK  →  { "alg": "HS256" }                  "Bu token HMAC-SHA256 ile imzalandı"
İÇERİK  →  { "sub": "murat",                    kime ait (kullanıcı adı)
             "iat": 1793270400,                  ne zaman üretildi
             "exp": 1793271300 }                 ne zaman geçersiz olacak (15 dk sonra)
İMZA    →  başlık + içerik + GİZLİ ANAHTAR karıştırılarak hesaplanan parmak izi
```

> **Çok önemli: JWT şifreli değildir!**
>
> Başlık ve içerik sadece Base64 ile kodlanmıştır. Token'ı jwt.io sitesine yapıştıran herkes içeriği okuyabilir. Bu yüzden içine **asla** şifre veya kart numarası gibi bilgiler konmaz. JWT'nin gücü gizlilikten değil, **imzadan** gelir.

2.4 İmza neden önemli?

Kötü niyetli biri token'ın içeriğini değiştirip `"sub": "murat"` yerine `"sub": "admin"` yazsa ne olur?

- İmza, **sadece sunucunun bildiği gizli anahtarla** hesaplanıyor (`application.yaml` içindeki `secret`).
- İçerik değişince imza artık tutmaz. Saldırgan yeni imzayı hesaplayamaz, çünkü gizli anahtarı bilmiyor.
- Sunucu imzayı kontrol eder, uyuşmadığını görür ve token'ı reddeder.

> **Benzetme**
>
> JWT, üzerinde noter mührü olan bir kâğıt gibidir. Kâğıttaki yazıyı herkes okuyabilir. Ama yazıyı değiştirirsen mühür bozulur, ve mührü sadece noter (sunucu) basabilir.

2.5 JWT'nin süper gücü ve zayıf noktası

✅ Süper gücü	❌ Zayıf noktası
Sunucu token'ı kontrol etmek için veritabanına bakmak zorunda değil. İmzaya ve süreye bakması yetiyor. Bu çok hızlı.	**Verilmiş bir JWT geri alınamaz.** Sunucu onu hiçbir yerde saklamadığı için "bu token'ı iptal ettim" diyemez. Süresi dolana kadar geçerlidir.

İşte refresh token tam bu zayıflığı kapatmak için var.

2.6 Refresh token nedir?

Refresh token **JWT değildir**. İçinde hiçbir bilgi olmayan, rastgele üretilmiş bir yazıdır:

```
qQxz_UIt3Kd9a-LmP0vW8sTnB2cR4eYh7jK1oX5uFgA      ← 43 karakter, tamamen rastgele
```

Kendi başına hiçbir anlamı yok. Anlamı sadece **veritabanındaki kayıtta**: "Bu yazı, şu kullanıcıya ait, şu tarihe kadar geçerli, iptal edilmemiş." Veritabanında durduğu için **istediğimiz an iptal edebiliriz**.

2.7 İkisi yan yana

	Access token	Refresh token
Biçimi	JWT (3 parçalı, içinde bilgi var)	Rastgele yazı (içinde bilgi yok)
Ömrü	**15 dakika**	**30 gün**
Ne işe yarar?	API'ye istek atmak	Sadece yeni access token almak
Nereye gider?	Her isteğin `Authorization` başlığına	Sadece `/auth/refresh` ve `/auth/logout` isteğinin gövdesine
Sunucu nasıl kontrol eder?	İmza ve süre (matematik)	Veritabanında arayarak
İptal edilebilir mi?	Hayır, süresi dolana kadar geçerli	Evet, veritabanında işaretlenir
Sunucuda saklanır mı?	Hayır	Sadece **hash'i** saklanır (bkz. 2.8)

2.8 Hash nedir?

Hash, bir yazıyı geri dönüşü olmayan şekilde başka bir yazıya çeviren matematiksel bir işlemdir.

```
SHA-256("qQxz_UIt3Kd9a-Lm...")  →  "7d2e9c01ab...41f0"   (her zaman 64 karakter)
```

- **Tek yönlüdür:** `7d2e9c01...`'den geriye `qQxz_...`'yi hesaplamak imkânsızdır.
- **Hep aynı sonucu verir:** Aynı yazı her seferinde aynı hash'i üretir. Bu sayede veritabanında arama yapabiliyoruz.

> **Neden token'ın kendisini değil de hash'ini saklıyoruz?**
> Veritabanı bir gün çalınırsa, hırsızın eline sadece hash'ler geçer. Hash'ten asıl token hesaplanamadığı için hırsız bunlarla `/refresh` isteği atamaz. Şifreleri de aynı sebeple hash'leyerek (BCrypt) saklıyoruz.

2.9 Küçük sözlük

Kelime	Basitçe
401 Unauthorized	"Seni tanımıyorum." Token yok, sahte veya süresi dolmuş.
Endpoint	Sunucudaki bir adres, örneğin `/api/expenses`.
Header (başlık)	İsteğin üstüne eklenen ek bilgi. Token `Authorization` başlığında gider.
Body (gövde)	İsteğin içindeki asıl veri (JSON). Refresh token burada gider.
Rotation	Her refresh'te eski refresh token'ı iptal edip yenisini vermek.
Revoke	İptal etmek. Veritabanında `revoked_at` alanını doldurmak.
TTL	Time to live, yaşam süresi.
SecureStore	Telefonun şifreli kasası (iPhone'da Keychain, Android'de Keystore).

3. Projenin haritası: kim ne yapar?



Tek cümleyle her parça

Parça	Ne yapar? (salağa anlatır gibi)
BE `application.yaml`	Ayar dosyası: gizli anahtar, token süreleri.
BE `SecurityConfig`	Kural kitabı: "Giriş kapıları herkese açık, geri kalan her yer token ister."
BE `JwtService`	Access token basan ve doğrulayan makine.
BE `JwtAuthenticationFilter`	Kapı görevlisi: her istekte token'a bakar, geçerliyse kişiye "bileklik" takar.
BE `CustomUserDetailsService`	"Bu kullanıcı hâlâ var mı ve aktif mi?" diye DB'ye bakar.
BE `CurrentUserService`	"Bu isteği kim attı?" sorusunun cevabını verir (harcamalar kime ait?).
BE `RefreshToken` (entity)	Refresh token tablosunun Java hali: hash, kullanıcı, bitiş, iptal tarihi.
BE `RefreshTokenRepository`	Tabloyla konuşur: "bu hash'i bul", "bu kullanıcının hepsini iptal et".
BE `RefreshTokenService`	Refresh token'ın her şeyi: üret, kontrol et, yenile, iptal et.
BE `AuthService`	Resepsiyon müdürü: şifreyi kontrol eder, iki token'ı birlikte verir.
BE `AuthController`	Resepsiyon masası: dışarıdan gelen 4 isteği karşılar.
BE `GlobalExceptionHandler`	Hataları düzgün cevaplara çevirir (ör. geçersiz refresh token → 401).
MOB `config.ts`	Backend'in adresi.
MOB `models/auth.ts`	Backend'le konuşurken kullanılan veri şekilleri.
MOB `tokenStorage.ts`	Telefonun şifreli kasası: iki token'ı saklar.
MOB `authService.ts`	Giriş, kayıt, yenileme ve çıkış isteklerini atar.
MOB `apiClient.ts`	Bütün isteklerin geçtiği kapı: token ekler, 401 gelince sessizce yeniler.
MOB `AuthContext.tsx`	Oturumun beyni: giriş yapıldı mı, token'ları yönet, çıkış yap.
MOB `_layout.tsx`, `index.tsx`	Giriş yapılmışsa ana sayfayı, yapılmamışsa login ekranını gösterir.

4. Backend sınıfları tek tek

Her sınıf için: **ne olduğu**, **gerçek kodu** ve **ne zaman çalıştığı**. Kodlar projedeki halinden kısaltılmıştır; asıl dosyalarda daha fazla yorum satırı var.

4.1 application.yaml: ayarlar

Benzetme: otelin kural panosu. "Kartlar 15 dakika, belgeler 30 gün geçerli."

`wallet-be/src/main/resources/application.yaml`

```
app:
  jwt:
    secret: ${JWT_SECRET:N7LpnYfTv5TAw349yiPcFn8MlnojcKZTjHOPsmdEVfM=}   # imza anahtarı
    access-token-ttl: 15m      # access token 15 dakika yaşar
  refresh-token:
    ttl: 30d                   # refresh token 30 gün yaşar
```

- **secret:** JWT'yi imzalayan gizli anahtar. Bunu bilen herkes sahte token üretebilir, o yüzden gerçek ortamda `JWT_SECRET` ortam değişkeninden gelir.
- **15m, 30d:** Spring bu yazıları otomatik olarak Java'nın `Duration` tipine çevirir. `30s` = 30 saniye, `24h` = 24 saat.

> **Test ipucu**
>
> Yenileme akışını denemek için `access-token-ttl: 30s` yapabilirsin. Test bitince **mutlaka** `15m` 'ye geri çek. 30 saniyede refresh tablosu çok hızlı dolar.

4.2 SecurityConfig: kural kitabı

Benzetme: otelin güvenlik planı. "Lobi herkese açık, odalar kart ister. Kapı görevlisi şurada dursun."

`wallet-be/src/main/java/com/fin/wallet/config/SecurityConfig.java`

```
@Bean
public PasswordEncoder passwordEncoder() {
    return new BCryptPasswordEncoder();          // şifreleri hash'leyen araç
}

@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http
        .csrf(csrf -> csrf.disable())            // cookie kullanmıyoruz, CSRF bize işlemez
        .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))  // sunucu kimseyi hatırlamaz
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/api/auth/**").permitAll()     // ① giriş kapıları herkese açık
            .requestMatchers("/error").permitAll()
            .anyRequest().authenticated())                   // ② geri kalan her şey kimlik ister
        .exceptionHandling(e -> e
            .authenticationEntryPoint(new HttpStatusEntryPoint(UNAUTHORIZED)))  // ③ kimlik yoksa 401
        .addFilterBefore(jwtAuthenticationFilter, UsernamePasswordAuthenticationFilter.class); // ④
    return http.build();
}
```

1. `/api/auth/**` **herkese açık:** login, register, refresh ve logout token istemez. Mantıklı: Giriş yapmak için zaten token'ın yok. Refresh'e de token'ın süresi dolduğu için geliyorsun.

2. **Geri kalan her şey kimlik ister:** `/api/expenses` gibi adreslere token'sız gelinemez.
3. **401 kararı burada verilir.** Kimliği olmayan biri korumalı bir adrese gelirse 401 döner. Mobil bu 401'i görünce token'ı yeniler.
4. **Kapı görevlisini yerleştir:** `JwtAuthenticationFilter` her istekte, kurallar kontrol edilmeden **önce** çalışsın.

Ne zaman çalışır? Uygulama açılırken bir kez okunur. Kurallar her istekte uygulanır.

4.3 JwtService: access token makinesi

Benzetme: oda kartı basan ve kartın sahte olup olmadığını anlayan makine.

`wallet-be/src/main/java/com/fin/wallet/security/JwtService.java`

```
public JwtService(@Value("${app.jwt.secret}") String secret,
                  @Value("${app.jwt.access-token-ttl}") Duration accessTokenTtl) {
    this.key = Keys.hmacShaKeyFor(Decoders.BASE64.decode(secret));   // yazıdan imza anahtarı üret
    this.accessTokenTtl = accessTokenTtl;                             // 15 dakika
}

// TOKEN ÜRET: login, register ve refresh'te çağrılır
public String generateToken(String userName) {
    Date now = new Date();
    return Jwts.builder()
            .subject(userName)                                          // "sub": "murat"
            .issuedAt(now)                                              // "iat": şimdi
            .expiration(new Date(now.getTime() + accessTokenTtl.toMillis())) // "exp": şimdi + 15 dk
            .signWith(key)                                              // gizli anahtarla imzala
            .compact();                                                 // "xxx.yyy.zzz" yazısına çevir
}

// TOKEN DOĞRULA: her istekte filtre çağırır
public String extractUserName(String token) {
    return Jwts.parser()
            .verifyWith(key)              // imzayı bu anahtarla kontrol et
            .build()
            .parseSignedClaims(token)     // ← ASIL KONTROL: imza bozuksa veya süre dolmuşsa EXCEPTION
            .getPayload()
            .getSubject();                // her şey yolundaysa kullanıcı adını döndür
}
```

"Süre dolmuş mu?" kontrolünü kim yapıyor?

Senin yazdığın bir `if` yok. `parseSignedClaims` token'ın içindeki `exp` alanına bakar. Süre geçmişse kendiliğinden `ExpiredJwtException` fırlatır. İmza tutmuyorsa da başka bir exception fırlatır. Yani bu tek satır hem "sahte mi?" hem de "süresi dolmuş mu?" sorularını cevaplıyor.

`parseSignedClaims` içeride sırayla ne yapıyor?

```
1. Token'ı üç parçaya ayır:   başlık . içerik . imza
2. İMZA kontrolü:  başlık + içerik + gizli anahtar → imzayı yeniden hesapla → gelenle aynı mı?
3. SÜRE kontrolü:  içerikteki "exp" alanı şu anki saatten küçük mü?
4. Her şey yolundaysa içeriği döndür
```

İmza, süreden **önce** kontrol ediliyor. Biri `exp` değerini ileri bir tarihe çekmeye çalışsa bile imza bozulduğu için daha süreye bakılmadan reddedilir. Token'ın süresini uzatmanın tek yolu gizli anahtarı bilmek.

Bu metot hangi durumlarda hata fırlatır?

Durum	Exception	Örnek
Süresi dolmuş	`ExpiredJwtException`	15 dakika (testte 30 saniye) geçti
İmza tutmuyor	`SignatureException`	İçerik değiştirildi; ya da `secret` değişti (o zaman eski token'ların **hepsi** geçersiz olur)
Biçim bozuk	`MalformedJwtException`	`"abc"`, yarım token, **refresh token'ı header'a koymak**
İmzasız token	`UnsupportedJwtException`	Başlığında `"alg":"none"` olan sahte token (tarihte yaşanmış ünlü bir saldırı)
Base64 çözülemiyor	`DecodingException`	Token'a geçersiz karakterler karışmış

Hepsi `JwtException`'ın alt sınıfı. Filtredeki tek bir `catch (JwtException ...)` hepsini yakalıyor ve her durumda sonuç aynı: bileklik takılmaz → 401.

> **Hatanın "süre doldu" olduğunu nasıl anlıyoruz? Anlamıyoruz, gerek de yok.**
> Ne backend ne de mobil 401'in sebebini ayırt ediyor. Mobil her durumda refresh deniyor. Sebep süre dolması, bozuk token veya değişen `secret` ise refresh başarılı olur ve sorun kendiliğinden düzelir. Kullanıcı banlandıysa veya refresh token da ölüyse refresh başarısız olur ve login ekranı açılır. İki sonuç da doğru.

Bu metodun bilmediği şeyler: Kullanıcı silindi mi, banlandı mı, çıkış yaptı mı? İlk ikisini filtrenin bir sonraki adımı (`loadUserByUsername` ve `isEnabled()`) yakalıyor. Üçüncüsünü hiçbir şey yakalamıyor: Çıkış yapan kullanıcının access token'ı süresi dolana kadar geçerli kalıyor.

4.4 JwtAuthenticationFilter: kapı görevlisi

Benzetme: konser kapısındaki görevli. Bileti geçerliyse bileğine bileklik takar, değilse takmaz. Ama kimseyi kovmaz, herkesi içeri yürütür. İçerideki kontrolcüler (SecurityConfig) bilekliği olmayanı salona sokmaz.

`wallet-be/src/main/java/com/fin/wallet/security/JwtAuthenticationFilter.java`

```
protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response,
                                FilterChain filterChain) throws ServletException, IOException {

    String header = request.getHeader("Authorization");        // "Bearer eyJhbGci..."

    if (header == null || !header.startsWith("Bearer ")) {     // token yoksa (ör. login isteği)
        filterChain.doFilter(request, response);               //   hiçbir şey yapmadan devam
        return;
    }

    String token = header.substring("Bearer ".length());       // "eyJhbGci..." kısmını ayır
    try {
        String userName = jwtService.extractUserName(token);   // ① imza + süre kontrolü
        UserDetails userDetails = userDetailsService.loadUserByUsername(userName); // ② DB'de var mı?

        if (userDetails.isEnabled()) {                          // ③ aktif mi?
            var authentication = new UsernamePasswordAuthenticationToken(
                    userDetails, null, userDetails.getAuthorities());
            SecurityContextHolder.getContext().setAuthentication(authentication);  // ④ BİLEKLİK TAK
        }
    } catch (JwtException | UsernameNotFoundException ex) {
        // sahte, süresi dolmuş veya silinmiş kullanıcı: bileklik takılmaz, ama kovulmaz da
    }

    filterChain.doFilter(request, response);                    // HER DURUMDA devam et
}
```

Gelen	Filtre ne yapar?	Sonra ne olur? (`/api/expenses` için)
Token yok	Bileklik takmaz	SecurityConfig → 401
Geçerli token	Bileklik takar	✅ Controller'a gider
Süresi dolmuş token	Exception → catch → bileklik takmaz	SecurityConfig → 401 → **mobil yeniler**
Değiştirilmiş (sahte) token	Exception → catch → bileklik takmaz	401
Refresh token'ı header'a koymak	JWT değil, çözülemez → bileklik takmaz	401. Refresh token API'ye girmeye yaramaz.

Bu projeye özel bir detay

Genel JWT anlatımlarında "access token kontrolü veritabanına gitmez" denir. Bu projede ise ② numaralı satır her istekte kullanıcıyı DB'den çekiyor. Bedeli küçük bir sorgu. Faydası şu: Bir kullanıcıyı pasife alırsan (`active = false`) etkisi **anında** olur. Elindeki token'ın süresinin dolmasını beklemek gerekmez. (Dosyadaki yorumda hâlâ "Token 24 saat geçerli" yazıyor. Bu artık 15 dakika.)

4.5 CustomUserDetailsService: "bu kullanıcı hâlâ var mı?"

Benzetme: resepsiyonun misafir listesi. Görevli kartı okuttuktan sonra listeye bakar: "Bu misafir hâlâ otelde mi, kara listede mi?"

`wallet-be/src/main/java/com/fin/wallet/security/CustomUserDetailsService.java`

```
public UserDetails loadUserByUsername(String username) {
    User user = userRepository.findByUserName(username)
            .orElseThrow(() -> new UsernameNotFoundException("User not found: " + username));

    return org.springframework.security.core.userdetails.User.builder()   // Spring'in anladığı biçim
            .username(user.getUserName())
            .password(user.getPassword())
            .authorities(List.of())
            .disabled(!user.isActive())      // pasif kullanıcı → isEnabled() false döner
            .build();
}
```

4.6 CurrentUserService: "bu isteği kim attı?"

Benzetme: görevlinin taktığı bilekliğe bakıp "sen 305 numaralı odadaki Murat'sın" demek.

`wallet-be/src/main/java/com/fin/wallet/security/CurrentUserService.java`

```
public User getCurrentUser() {
    Authentication authentication = SecurityContextHolder.getContext().getAuthentication(); // bilekliği oku
    return userRepository.findByUserName(authentication.getName())   // bileklikteki ad → kullanıcı
            .orElseThrow(...);
}
```

Harcama servisleri bunu kullanır: "Harcamaları getir ama **sadece bu kullanıcınınkileri**." Kullanıcının kim olduğu, mobilin gönderdiği bir id'den değil **token'dan** gelir. Böylece kimse başkasının id'sini yazıp onun harcamalarını göremez.

4.7 RefreshToken: tablonun Java hali

Benzetme: resepsiyonun belge defterindeki bir satır.

`wallet-be/src/main/java/com/fin/wallet/entity/RefreshToken.java`

```
@Entity
@Table(name = "refresh_tokens")
public class RefreshToken extends BaseEntity {       // id, created_at, updated_at buradan gelir

    @Column(name = "token_hash", nullable = false, unique = true, length = 64)
    private String tokenHash;            // token'ın KENDİSİ DEĞİL, SHA-256 hash'i

    @ManyToOne(fetch = FetchType.LAZY, optional = false)
    @JoinColumn(name = "user_id", nullable = false)
    private User user;                   // kime ait (bir kullanıcının birden çok cihazı = birden çok satır)

    @Column(name = "expires_at", nullable = false)
    private LocalDateTime expiresAt;     // ne zamana kadar geçerli (oluşturulma + 30 gün)

    @Column(name = "revoked_at")
    private LocalDateTime revokedAt;     // null = AKTİF, dolu = İPTAL EDİLMİŞ
}
```

Veritabanında şöyle görünür:

id	user_id	token_hash	expires_at	revoked_at	Anlamı
4	2	7d2e9c...41f0	07.12 14:10	07.11 14:12	Kullanılmış, iptal
5	2	a91b03...77c2	07.12 14:12	*(boş)*	✅ Aktif

Neden kullanılan token'ı silmiyoruz da `revoked_at` dolduruyoruz?

Çünkü eski token'ın kaydı defterde kalmalı ki biri onu tekrar getirdiğinde "bu zaten kullanılmıştı, çalınmış!" diyebilelim. Silseydik "böyle bir belge yok" der ve hırsızı fark etmezdik.

4.8 RefreshTokenRepository: tabloyla konuşan

Benzetme: defteri karıştıran memur.

`wallet-be/src/main/java/com/fin/wallet/repository/RefreshTokenRepository.java`

```
public interface RefreshTokenRepository extends JpaRepository<RefreshToken, Long> {

    // "Bu hash'e sahip satırı bul" (Spring metot adından sorguyu kendisi üretir)
    Optional<RefreshToken> findByTokenHash(String tokenHash);

    // "Bu kullanıcının aktif bütün token'larını tek seferde iptal et"
    @Modifying
    @Query("UPDATE RefreshToken rt SET rt.revokedAt = :now " +
           "WHERE rt.user.id = :userId AND rt.revokedAt IS NULL")
    int revokeAllActiveByUserId(@Param("userId") Long userId, @Param("now") LocalDateTime now);
}
```

- `findByTokenHash` : Gövdesini yazmıyorsun. Spring metot adını okuyup `SELECT ... WHERE token_hash = ?` sorgusunu kendisi üretiyor.
- `revokeAllActiveByUserId` : Çalıntı token yakalandığında ve kullanıcı banlandığında kullanılıyor.

4.9 RefreshTokenService: refresh token'ın beyni

Benzetme: resepsiyonda belge işlerine bakan görevli. Belge verir, belgeyi kontrol eder, eskisini yırtıp yenisini verir, iptal eder.

`wallet-be/src/main/java/com/fin/wallet/service/RefreshTokenService.java`

a) generateRawToken(): rastgele token üretmek

```
private static final SecureRandom SECURE_RANDOM = new SecureRandom();

private String generateRawToken() {
    byte[] bytes = new byte[32];                                    // 32 byte = 256 bit
    SECURE_RANDOM.nextBytes(bytes);                                 // rastgele doldur
    return Base64.getUrlEncoder().withoutPadding().encodeToString(bytes); // yazıya çevir (43 karakter)
}
```

`SecureRandom` tahmin edilemeyen rastgele sayılar üretir (normal `Random` tahmin edilebilir). 256 bit, yaklaşık 10^{77} ihtimal demek: Evrendeki atom sayısına yakın. Kimse tahmin edemez.

b) hash(): SHA-256

```
private String hash(String rawToken) {
    MessageDigest digest = MessageDigest.getInstance("SHA-256");
    return HexFormat.of().formatHex(digest.digest(rawToken.getBytes(UTF_8)));   // 64 karakter
}
```

Neden şifrelerdeki gibi BCrypt değil?

BCrypt her seferinde farklı sonuç verir. O zaman veritabanında "bu token'ı bul" diye arama yapamazdık. SHA-256 ise hep aynı sonucu verir. Şifreler zayıf olduğu için (`123456` gibi) yavaş ve tuzlu bir hash olan BCrypt gerekir. Refresh token ise 256 bit rastgele olduğu için tahmin edilemez, hızlı SHA-256 yeterli.

c) create(user): yeni refresh token (login, register ve her refresh'te)

```
@Transactional
public String create(User user) {
    String rawToken = generateRawToken();                       // ① rastgele token üret

    RefreshToken refreshToken = new RefreshToken();
    refreshToken.setTokenHash(hash(rawToken));                  // ② DB'ye HASH'i yaz
    refreshToken.setUser(user);
    refreshToken.setExpiresAt(LocalDateTime.now().plus(ttl));   //    şimdi + 30 gün
    refreshTokenRepository.save(refreshToken);                  // ③ tabloya YENİ SATIR

    return rawToken;                                            // ④ ham token'ı istemciye ver
}
```

Dikkat

Ham token sadece bu metodun içinde birkaç milisaniye yaşar. DB'ye hash'i gider, ham hali telefona gider. Sunucuda hiçbir yerde saklanmaz.

d) rotate(rawToken): kontrol et, eskisini iptal et, yenisini ver (/refresh)

```
@Transactional(noRollbackFor = InvalidRefreshTokenException.class)
public RotationResult rotate(String rawToken) {
    // ① Defterde var mı?
    RefreshToken current = refreshTokenRepository.findByTokenHash(hash(rawToken))
            .orElseThrow(InvalidRefreshTokenException::new);              // yok → 401

    LocalDateTime now = LocalDateTime.now();
    User user = current.getUser();

    // ② Daha önce kullanılmış mı? → ÇALINTI!
    if (current.getRevokedAt() != null) {
        refreshTokenRepository.revokeAllActiveByUserId(user.getId(), now); // bütün oturumları kapat
        throw new InvalidRefreshTokenException();                          // 401
    }

    // ③ 30 günü dolmuş mu?
    if (current.getExpiresAt().isBefore(now)) {
        throw new InvalidRefreshTokenException();                          // 401
    }

    // ④ Kullanıcı banlanmış mı?
    if (!user.isActive()) {
        refreshTokenRepository.revokeAllActiveByUserId(user.getId(), now);
        throw new InvalidRefreshTokenException();                          // 401
    }

    // ⑤ Her şey yolunda: eskisini iptal et, yenisini üret
    current.setRevokedAt(now);                    // save() gerekmez, Hibernate değişikliği fark eder
    return new RotationResult(user, create(user)); // tabloya YENİ SATIR
}
```

> **`noRollbackFor` ne işe yarıyor?**
>
> Normalde bir metot hata fırlatınca Spring o metotta yapılan bütün veritabanı işlemlerini **geri alır**. ② numaralı adımda önce bütün oturumları iptal edip sonra hata fırlatıyoruz. `noRollbackFor` olmasaydı Spring bu iptalleri de geri alırdı ve hırsız içeride kalırdı. Bu ayar "Bu hatada geri alma yapma" demek.

> **Neden ② (kullanılmış mı?), ③'ten (süresi dolmuş mu?) önce?**
>
> Sıra tersi olsaydı, süresi dolmuş çalıntı bir token sadece "süresi dolmuş" diye sessizce reddedilirdi ve hırsızlık hiç fark edilmezdi.

e) revoke(rawToken): çıkış (/logout)

```
@Transactional
public void revoke(String rawToken) {
    refreshTokenRepository.findByTokenHash(hash(rawToken))
            .filter(token -> token.getRevokedAt() == null)            // hâlâ aktifse
            .ifPresent(token -> token.setRevokedAt(LocalDateTime.now())); // iptal et
    // bulunamazsa HİÇBİR ŞEY YAPMA: çıkış her zaman başarılıdır
}
```

f) revokeAll(userId): bütün cihazlardan çıkış

```
@Transactional
public void revokeAll(Long userId) {
    refreshTokenRepository.revokeAllActiveByUserId(userId, LocalDateTime.now());
}
```

Şimdilik hiçbir yerden çağrılmıyor. İleride "Tüm cihazlardan çıkış yap" butonu veya şifre değiştirme özelliği eklendiğinde kullanılacak.

4.10 AuthService: resepsiyon müdürü

Benzetme: kimliğe bakan, kartı ve belgeyi birlikte veren müdür. JwtService ile RefreshTokenService'i bir araya getirir.

wallet-be/src/main/java/com/fin/wallet/service/AuthService.java

```
// GİRİŞ
@Transactional
public AuthResponse login(LoginRequest request) {
    User user = userRepository.findByUserName(request.userName())
            .orElseThrow(() -> new BadCredentialsException(INVALID_CREDENTIALS));  // böyle biri yok → 401

    if (!user.isActive() || !passwordEncoder.matches(request.password(), user.getPassword())) {
        throw new BadCredentialsException(INVALID_CREDENTIALS);                  // şifre yanlış → 401
    }
    return issueTokens(user);                                                    // iki token ver
}

// KAYIT (kullanıcıyı kaydeder, sonra aynı şekilde iki token verir)
@Transactional
public AuthResponse register(RegisterRequest request) { ... return issueTokens(user); }

// YENİLEME (bilerek @Transactional DEĞİL, bkz. aşağıdaki kutu)
public AuthResponse refresh(RefreshRequest request) {
    RotationResult result = refreshTokenService.rotate(request.refreshToken());
    return new AuthResponse(
            jwtService.generateToken(result.user().getUserName()),   // yeni access token
            result.refreshToken());                                   // rotate'in ürettiği yeni refresh token
}

// ÇIKIŞ
public void logout(RefreshRequest request) {
    refreshTokenService.revoke(request.refreshToken());
}

// Login ve register'ın ortak sonu: iki token birden
private AuthResponse issueTokens(User user) {
    return new AuthResponse(
            jwtService.generateToken(user.getUserName()),    // 15 dakikalık JWT
            refreshTokenService.create(user));               // 30 günlük refresh token (DB'ye yazılır)
}
```

Yanlış şifrede neden "kullanıcı adı veya şifre hatalı" diyoruz?

"Böyle bir kullanıcı yok" deseydik, saldırgan hangi kullanıcı adlarının kayıtlı olduğunu öğrenirdi. Hangisinin yanlış olduğunu bilerek söylemiyoruz.

`refresh()` neden `@Transactional` değil?

Olsaydı `rotate()` kendi transaction'ını açmaz, bu dıştaki transaction'a katılırdı. Çalıntı token yakalandığında fırlatılan hata dıştaki metottan geçerken dıştaki transaction her şeyi geri alırdı. `rotate` 'teki `noRollbackFor` boşa giderdi ve hırsız içeride kalırdı.

4.11 AuthController ve DTO'lar: resepsiyon masası

Benzetme: dışarıdan gelenleri karşılayan masa. Kendisi karar vermez, işi müdüre (AuthService) iletir.

wallet-be/src/main/java/com/fin/wallet/controller/AuthController.java

```
@RestController
@RequestMapping("/api/auth")
public class AuthController {

    @PostMapping("/login")        // giriş → 200 + iki token
    public ResponseEntity<AuthResponse> login(@Valid @RequestBody LoginRequest request) {
        return ResponseEntity.ok(authService.login(request));
    }

    @PostMapping("/register")     // kayıt → 201 + iki token
    public ResponseEntity<AuthResponse> register(@Valid @RequestBody RegisterRequest request) {
        return ResponseEntity.status(HttpStatus.CREATED).body(authService.register(request));
    }

    @PostMapping("/refresh")      // yenileme → 200 + YENİ iki token
    public ResponseEntity<AuthResponse> refresh(@Valid @RequestBody RefreshRequest request) {
        return ResponseEntity.ok(authService.refresh(request));
    }

    @PostMapping("/logout")       // çıkış → 204 (gövdesiz)
    public ResponseEntity<Void> logout(@Valid @RequestBody RefreshRequest request) {
        authService.logout(request);
        return ResponseEntity.noContent().build();
    }
}
```

`dto/auth/*.java: istek ve cevapların şekli`

```
public record LoginRequest(@NotBlank String userName, @NotBlank String password) { }
public record RefreshRequest(@NotBlank String refreshToken) { }
public record AuthResponse(String accessToken, String refreshToken) { }
```

Mobil ile backend arasında giden JSON'lar:

```
POST /api/auth/login     {"userName":"murat","password":"..."}
                    ←    {"accessToken":"eyJhbGci...","refreshToken":"qQxz_UIt..."}

POST /api/auth/refresh   {"refreshToken":"qQxz_UIt..."}
                    ←    {"accessToken":"eyJ...YENİ","refreshToken":"Lm9a_...YENİ"}

POST /api/auth/logout    {"refreshToken":"Lm9a_..."}
                    ←    204 (boş)
```

4.12 InvalidRefreshTokenException ve GlobalExceptionHandler

Benzetme: "Belgen geçersiz" diyen tek bir standart cevap.

```
public class InvalidRefreshTokenException extends RuntimeException {
    public InvalidRefreshTokenException() { super("Invalid refresh token"); }
}

// GlobalExceptionHandler içinde:
@ExceptionHandler(InvalidRefreshTokenException.class)
public ProblemDetail handleInvalidRefreshToken(InvalidRefreshTokenException ex) {
    return ProblemDetail.forStatusAndDetail(HttpStatus.UNAUTHORIZED, ex.getMessage());   // 401
}
```

Dört farklı durum var: bulunamadı, kullanılmış, süresi dolmuş ve kullanıcı pasif. Hepsinde **aynı mesaj** dönüyor. Saldırgan hangi durumda olduğunu öğrenemez. Mobil bu 401'i görünce "oturum bitti, login ekranına git" kararını

verir.

5. Mobil dosyalar tek tek

5.1 config.ts: backend nerede?

`wallet-mobile/src/constants/config.ts`

```
export const API_BASE_URL = resolveApiBaseUrl();   // ör. http://192.168.1.20:8080/api
export const REQUEST_TIMEOUT_MS = 15000;           // 15 saniyede cevap gelmezse vazgeç
```

Telefonda `localhost` telefonun kendisi demektir. Bu yüzden bilgisayarın ağ IP'si kullanılır.

5.2 models/auth.ts: veri şekilleri

`wallet-mobile/src/models/auth.ts`

```
export interface AuthResponse {
  accessToken: string;    // 15 dk, her isteğin başlığına eklenir
  refreshToken: string;   // 30 gün, sadece /refresh ve /logout'a gider
}
export interface RefreshRequest {
  refreshToken: string;
}
```

Backend'deki Java record'larının TypeScript'teki aynası. Alan adları birebir aynı olmalı, yoksa JSON eşleşmez.

5.3 tokenStorage.ts: telefonun kasası

Benzetme: cüzdanın. Kartı ve belgeyi birlikte koyarsın, birlikte çıkarırsın.

`wallet-mobile/src/services/tokenStorage.ts`

```
const ACCESS_TOKEN_KEY = 'auth_token';
const REFRESH_TOKEN_KEY = 'auth_refresh_token';

export const tokenStorage = {
  // İkisinden biri eksikse "oturum yok" say
  async get(): Promise<StoredTokens | null> {
    const [accessToken, refreshToken] = await Promise.all([
      getItem(ACCESS_TOKEN_KEY), getItem(REFRESH_TOKEN_KEY)]);
    if (!accessToken || !refreshToken) return null;
    return { accessToken, refreshToken };
  },
  // İkisini BİRLİKTE yaz
  async set(tokens: StoredTokens) {
    await Promise.all([
      setItem(ACCESS_TOKEN_KEY, tokens.accessToken),
      setItem(REFRESH_TOKEN_KEY, tokens.refreshToken)]);
  },
  // İkisini BİRLİKTE sil
  async remove() {
    await Promise.all([removeItem(ACCESS_TOKEN_KEY), removeItem(REFRESH_TOKEN_KEY)]);
  },
};
```

- **iPhone / Android:** `SecureStore` kullanılır. İşletim sisteminin şifreli kasası (Keychain / Keystore). Başka uygulamalar okuyamaz.

- **Web:** SecureStore çalışmadığı için `localStorage` kullanılır. Daha az güvenli, bilinçli bir taviz.
- `Promise.all` : İki işlemi aynı anda başlatır, ikisinin de bitmesini bekler.

5.4 authService.ts: giriş kapısına giden istekler

wallet-mobile/src/services/authService.ts

```
const PUBLIC = { auth: false };    // bu isteklere access token EKLEME

export const authService = {
  login:    (req) => apiClient.post<AuthResponse>('/auth/login', req, PUBLIC),
  register: (req) => apiClient.post<AuthResponse>('/auth/register', req, PUBLIC),
  refresh:  (req) => apiClient.post<AuthResponse>('/auth/refresh', req, PUBLIC),
  logout:   (req) => apiClient.post<void>('/auth/logout', req, PUBLIC),
};
```

`{ auth: false }` neden önemli?

Bu isteklerde 401 gelirse anlamı farklı. Login'de 401 = "şifre yanlış". Refresh'te 401 = "oturum tamamen bitti". İkisinde de "token'ı yenileyip tekrar dene" yapılmamalı, yoksa sonsuz döngüye girilir. `auth: false` dendiğinde apiClient yenileme denemez.

5.5 apiClient.ts: bütün isteklerin geçtiği kapı (en önemli dosya)

Benzetme: sen. Kartını kapıya okutursun. Kapı açılmazsa resepsiyona koşup yeni kart alır, geri gelip tekrar okutursun. Ve bunu kimseye söylemezsin.

`wallet-mobile/src/services/api/apiClient.ts`

a) Hafızadaki değişkenler

```
let authToken: string | null = null;     // şu anki access token (her isteğe eklenir)
let unauthorizedHandler = null;          // "oturum kurtarılamadı" olunca çağrılacak (→ login ekranı)
let tokenRefresher = null;               // "token'ı yenile" diye çağrılacak (AuthContext verir)
let refreshPromise = null;               // ŞU AN devam eden yenileme (varsa)
```

b) send(): isteği bir kez göndermek

```
async function send(method, path, body, token) {
  const headers = { 'Content-Type': 'application/json' };
  if (token) {
    headers.Authorization = `Bearer ${token}`;    // ← token BURADA eklenir
  }
  return await fetch(`${API_BASE_URL}${path}`, { method, headers, body: JSON.stringify(body) });
  // (zaman aşımı ve "internet yok" hataları da burada ApiError'a çevrilir)
}
```

c) request(): asıl mantık (401 gelince yenile ve tekrar dene)

```
async function request(method, path, body, options = {}) {
  const token = options.auth === false ? null : authToken;     // hangi token'la gidiyoruz?

  let response = await send(method, path, body, token);        // ① İSTEĞİ GÖNDER

  if (response.status === 401 && token) {                      // ② token gönderdim ama 401 geldi
                                                               //    → access token'ın süresi dolmuş

    const newToken = authToken !== token                       // ③ başka bir istek bu arada
      ? authToken                                              //    zaten yeniledi mi? → onu kullan
      : await refreshAccessToken();                            //    yoksa YENİLE

    if (!newToken) {                                           // ④ yenilenemedi (refresh token da ölü)
      throw sessionExpired();                                  //    → login ekranı
    }

    response = await send(method, path, body, newToken);       // ⑤ AYNI İSTEĞİ yeni token'la TEKRAR gönder
    if (response.status === 401) {                             // ⑥ yine 401? sonsuz döngüye girme
      throw sessionExpired();
    }
  }

  if (!response.ok) throw await toApiError(response);          // diğer hatalar (400, 404, 500...)
  if (response.status === 204) return undefined;
  return await response.json();                                // ✅ başarılı cevap
}
```

d) refreshAccessToken(): aynı anda TEK yenileme

```
function refreshAccessToken() {
  if (!refreshPromise) {                                  // devam eden yenileme YOKSA
    refreshPromise = tokenRefresher().finally(() => {     //   yenisini başlat
      refreshPromise = null;                              //   bitince temizle
    });
  }
  return refreshPromise;                                  // varsa ONU döndür (herkes aynı sonucu bekler)
}
```

Neden aynı anda sadece bir yenileme?

Ana sayfa açılınca 3 istek birden gidiyor ve üçü de 401 alıyor diyelim. Üçü ayrı ayrı **aynı** refresh token'la `/refresh`'e giderse: 1. istek başarılı olur ve token'ı iptal eder. 2. istek **iptal edilmiş** token'la gelir. Backend bunu çalıntı sanır ve **bütün oturumlarını kapatır**. Sen hiçbir şey yapmadan login ekranına atılırsın. `refreshPromise` sayesinde 3 istek de tek bir yenilemenin sonucunu bekliyor.

5.6 AuthContext.tsx: oturumun beyni

Benzetme: kafandaki "otelde kalıyor muyum?" bilgisi ve cüzdanını yöneten sen.

`wallet-mobile/src/context/AuthContext.tsx`

a) Durum

```
const [token, setToken] = useState<string | null>(null);   // access token (ekranlar bunu bilir)
const [isLoading, setIsLoading] = useState(true);          // açılışta kasa okunurken true
const refreshTokenRef = useRef<string | null>(null);       // refresh token (ekranlar BİLMEZ)
```

Refresh token neden `useState` değil de `useRef`? Ekranlarda gösterilmiyor. State'te olsaydı her yenilemede bütün uygulama gereksiz yere yeniden çizilirdi. `ref` değişince ekran yeniden çizilmez.

b) startSession(): token'ları dört yere koy

```
const startSession = async (tokens) => {
  await tokenStorage.set(tokens);                  // ① kasa (uygulama kapansa da kalsın)
  refreshTokenRef.current = tokens.refreshToken;   // ② hafıza (yenilemede kullanılacak)
  setAuthToken(tokens.accessToken);                // ③ apiClient (her isteğe eklensin)
  setToken(tokens.accessToken);                    // ④ ekranlar ("giriş yapıldı" bilsin)
};
```

c) signIn / signUp

```
const signIn = async (request) => {
  await startSession(await authService.login(request));    // giriş yap → token'ları yerleştir
};
```

d) refreshSession(): apiClient "yenile!" deyince çalışan fonksiyon

```
const refreshSession = async () => {
  const refreshToken = refreshTokenRef.current;
  if (!refreshToken) return null;                                    // refresh token yok → oturum yok
  try {
    const tokens = await authService.refresh({ refreshToken });   // POST /auth/refresh
    if (refreshTokenRef.current !== refreshToken) return null;     // bu arada çıkış yapıldıysa kullanma
    await startSession(tokens);                                    // yeni token'ları yerleştir
    return tokens.accessToken;                                     // apiClient'a yeni access token'ı ver
  } catch (e) {
    if (e instanceof ApiError && e.status === 401) return null;    // refresh token ölü → oturum bitti
    throw e;                                                       // internet yok → oturumu KAPATMA
  }
};
```

İnternet yokken neden oturum kapatılmıyor?

Asansördesin, çekmiyor. Token'ın süresi dolmuş, yenileme denendi ama sunucuya ulaşılamadı. Bu, oturumun bittiği anlamına gelmez. Seni login ekranına atmak saçma olurdu. Sadece "Sunucuya bağlanılamadı" hatası gösterilir, internet gelince her şey devam eder.

e) signOut(): çıkış

```
const signOut = async () => {
  const refreshToken = refreshTokenRef.current;
  await clearSession();                        // ① önce telefonda temizle (anında login ekranı)
  if (refreshToken) {
    authService.logout({ refreshToken })       // ② sonra backend'e haber ver (beklemeden)
      .catch(() => undefined);                 //    başarısız olsa da sorun değil
  }
};
```

f) Açılışta kasayı oku ve apiClient'a yetki ver

```
useEffect(() => {                              // uygulama açılınca bir kez
  tokenStorage.get().then((stored) => {
    refreshTokenRef.current = stored?.refreshToken ?? null;
    setAuthToken(stored?.accessToken ?? null);
    setToken(stored?.accessToken ?? null);
  }).finally(() => setIsLoading(false));      // splash ekranı kapanabilir
}, []);

useEffect(() => {
  setTokenRefresher(refreshSession);           // apiClient: "yenilemen gerekirse BUNU çağır"
  setUnauthorizedHandler(() => clearSession()); // apiClient: "oturum ölürse BUNU çağır"
}, [refreshSession, clearSession]);
```

apiClient React'ten habersiz düz bir dosya. AuthContext ona bu iki fonksiyonu vererek "yenileme ve çıkış işini bana bırak" diyor.

g) Refresh isteği tam olarak nerede atılıyor? (dört dosyalık zincir)

```
apiClient.ts        refreshAccessToken()                  "yenileme lazım"
     │
apiClient.ts        tokenRefresher()                      "kayıtlı yenileyiciyi çağır"
     │                (bu aslında AuthContext'teki refreshSession, setTokenRefresher ile verildi)
     ▼
AuthContext.tsx     refreshTokenRef.current               "refresh token'ı hafızadan al"
AuthContext.tsx     authService.refresh({ refreshToken }) "isteği başlat"
     │
authService.ts      apiClient.post('/auth/refresh', body, { auth: false })
     │
apiClient.ts        send() → fetch(...)                   ← İSTEK BURADA GİDİYOR
```

```
POST http://192.168.x.x:8080/api/auth/refresh
Content-Type: application/json
                                   ← Authorization başlığı YOK ({ auth: false })
{"refreshToken":"qQxz_UIt3Kd9a-LmP0vW8sTnB2cR4eYh7jK1oX5uFgA"}
```

Neden bu kadar dolambaçlı?

apiClient refresh token'ı bilmiyor, o AuthContext'in `ref`'inde duruyor. Yeni token'ları kasaya kaydetmek ve ekranları güncellemek de AuthContext'in işi. İş bölümü şöyle: **apiClient "ne zaman"** yenileneceğini biliyor (401 geldiğinde, ve aynı anda tek sefer). **AuthContext "nasıl"** yenileneceğini biliyor (refresh token'ı al, isteği at, sonucu kaydet).

5.7 _layout.tsx ve index.tsx: hangi ekran gösterilsin?

wallet-mobile/src/app/_layout.tsx, src/app/index.tsx

```
// _layout.tsx
const { token, isLoading } = useAuth();
const signedIn = token !== null;
if (isLoading) return null;                    // kasa okunurken hiçbir şey çizme (splash önde)

<Stack.Protected guard={!signedIn}>            // SADECE giriş yapılmamışken açılabilir
  <Stack.Screen name="login" />
  <Stack.Screen name="register" />
</Stack.Protected>
<Stack.Protected guard={signedIn}>             // SADECE giriş yapılmışken açılabilir
  <Stack.Screen name="(drawer)" />             //   ana sayfa, harcamalar...
</Stack.Protected>

// index.tsx: açılışta yönlendirme
return <Redirect href={token ? '/home' : '/login'} />;
```

Hiçbir yerde "giriş başarılı, ana sayfaya git" diye elle yönlendirme yok. `token` dolunca bu kurallar kendiliğinden login'i kapatıp ana sayfayı açıyor. `token` null olunca (çıkış veya oturum bitti) tam tersi oluyor.

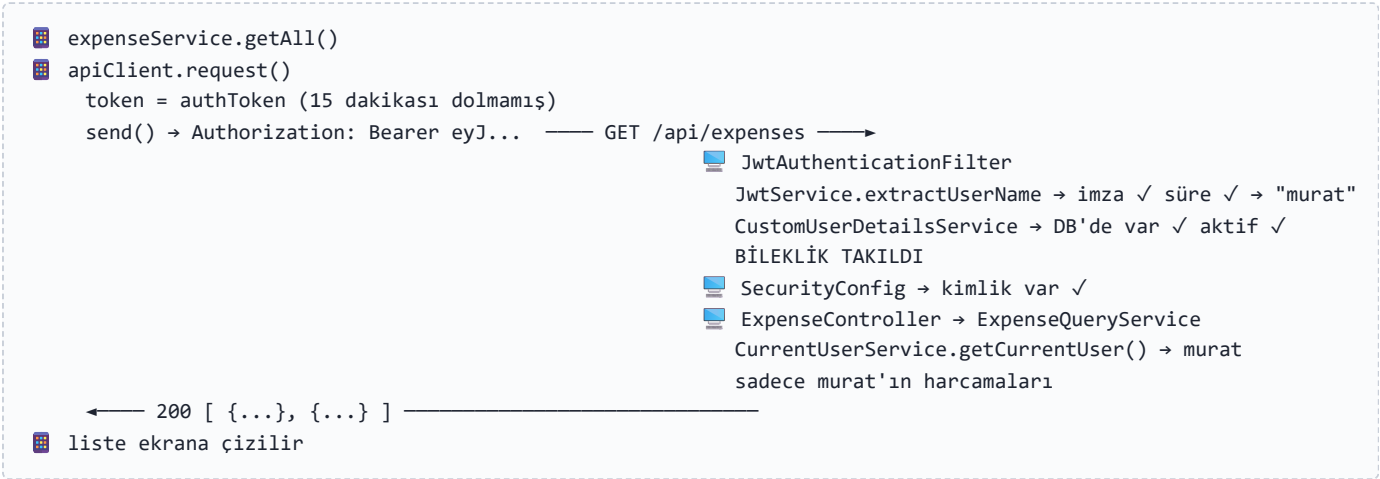
6. Senaryo: Giriş yapmak

Kullanıcı adını ve şifreyi yazıp "Giriş" butonuna bastın.



Şifre yanlışsa: Backend 401 döner. `auth: false` olduğu için apiClient yenileme denemez. `useLoginForm` hatayı yakalar ve "Kullanıcı adı veya şifre hatalı" yazar.

7. Senaryo: Normal bir istek (harcamaları listele)

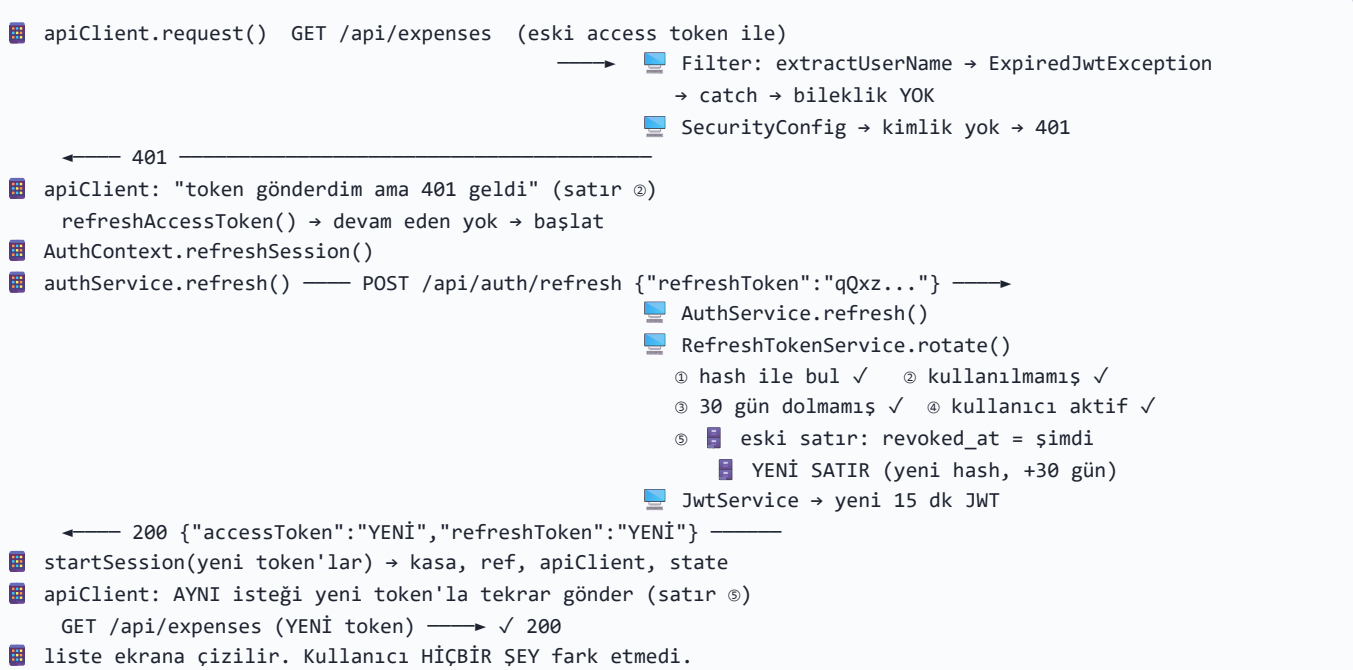


Bu senaryoda refresh_tokens tablosuna HİÇ dokunulmaz

Normal isteklerde sadece access token kullanılır. Tabloya satır eklenmez.

8. Senaryo: 15 dakika doldu (sessiz yenileme)

Giriş yapalı 20 dakika oldu. Harcamalar ekranını yeniledin.



Tabloya ne zaman satır eklenir?

Access token'ın süresi dolduktan sonra atılan ilk istekte bir satır eklenir. Zamanlayıcı yoktur. Uygulama açık ama sen hiç istek atmıyorsan, saatlerce beklesen de satır eklenmez. Süre dolduktan sonraki ilk istek 401 alır ve yenileme o zaman olur.

401'i kim üretiyor?

Filtre değil. Filtre sadece bilekliği takmıyor ve isteği yoluna devam ettiriyor. Spring Security, kimliği olmayan birinin `authenticated()` isteyen bir adrese geldiğini görüyor ve `SecurityConfig`'teki `HttpStatusEntryPoint` 401 döndürüyor. Controller hiç çalışmıyor.

Eski token'lara ne oldu?

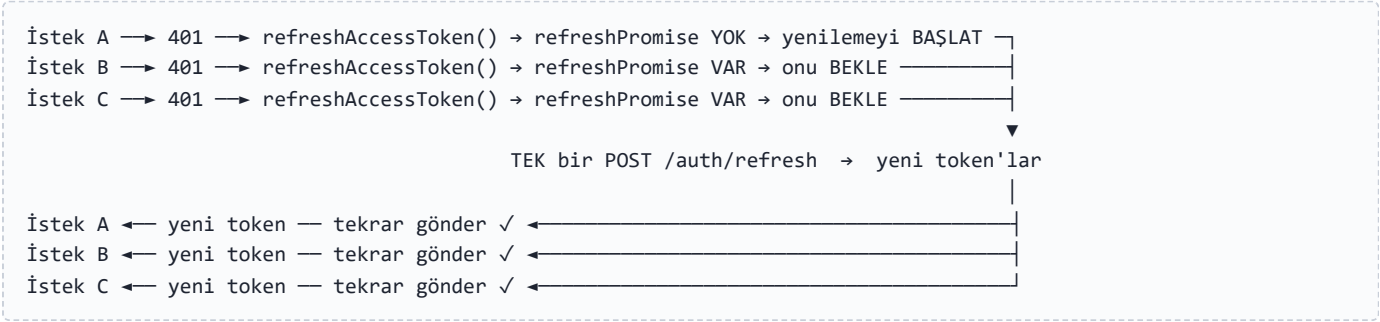
Token	Ne oldu?
AT1 (eski access token)	Hiçbir şey yapılmadı. Zaten süresi dolmuştu. JWT iptal edilemez, gerek de yok.
RT1 (eski refresh token)	DB'de **iptal edildi** (`revoked_at` doldu). Satır silinmedi. Biri tekrar getirirse "çalıntı" sayılır.
AT2, RT2 (yeniler)	Mobilin her yerine (kasa, ref, apiClient, state) eskilerin yerine yazıldı. RT2 yeni bir 30 gün ile geldi.

Refresh token neden her seferinde yenileniyor? (rotation)

- **Yenilenmeseydi:** Çalınan bir RT1 30 gün boyunca aynı kalır ve hırsızın kullandığı hiç fark edilmezdi, çünkü ikiniz de geçerli bir token kullanıyor olurdunuz.
- **Yenilendiği için:** Çalınan token, senin bir sonraki yenilemende bayatlıyor. İptal edilmiş token tekrar gelirse hırsızlık yakalanıyor. Bonus olarak düzenli kullanan biri hiç login ekranı görmüyor.

9. Senaryo: Aynı anda 3 istek

Ana sayfa açılıyor: harcamalar, özet ve kullanıcı bilgisi aynı anda isteniyor. Access token'ın süresi dolmuş.



Tabloya sadece **bir** satır eklenir. Bu mekanizma olmasaydı 3 ayrı refresh isteği gider, 2. ve 3. istek "çalıntı" sanılır ve kullanıcı dışarı atılırdı.

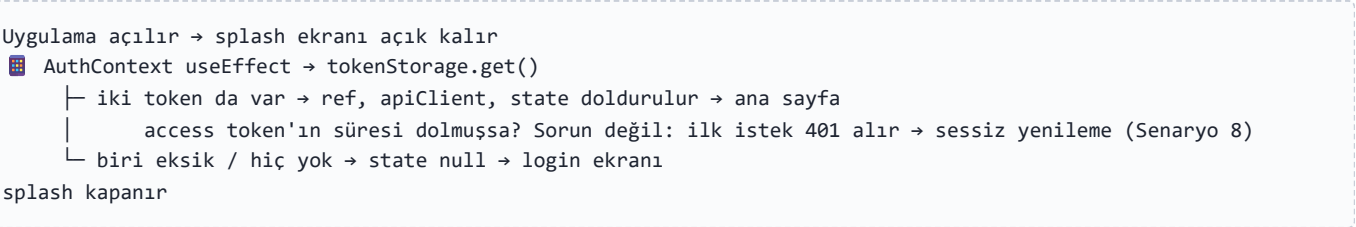
Bir de şu durum var: İstek D, yenileme **bittikten sonra** eski token'la 401 alırsa, ③ numaralı satır `authToken !== token` olduğunu görür. Yani "başkası zaten yenilemiş" der ve yeniden yenileme yapmadan hemen yeni token'la tekrar dener.

10. Senaryo: Çıkış yapmak



- **Neden önce telefonda temizliyoruz?** Kullanıcı beklemesin. İnternet yokken de çıkış yapılabilsin.
- **Backend'e ulaşılamazsa ne olur?** Refresh token telefonda silindi, kimse kullanamaz. DB'de 30 gün sonra kendiliğinden geçersiz olur.
- **Access token ne olur?** Telefondan silindi. Ama bir kopyası çalınmışsa en fazla 15 dakika daha çalışır, çünkü JWT geri alınamaz. Kısa tutmamızın sebebi bu.

11. Senaryo: Uygulamayı kapatıp açmak



30 gün hiç açmadıysan: Kasada token'lar var, ana sayfa açılır. İlk istek 401 alır ve yenileme denenir. Ama refresh token'ın 30 günü dolmuştur: `rotate()` ③ numaralı kontrolde 401 döner. `refreshSession` `null` döner, apiClient `sessionExpired()` çağırır ve login ekranı açılır.

Düzenli kullanıyorsan: Her yenilemede **yeni** bir refresh token, yani **yeni 30 gün** veriliyor. Bu yüzden uygulamayı kullanan biri hiç login ekranı görmez. Buna "kayan oturum" (sliding session) denir.

12. Senaryo: Token çalındı (tekrar kullanım tespiti)

Bir hırsız bir şekilde senin refresh token'ını (RT1) kopyaladı.

Durum A: Önce sen kullanıyorsun

```
📱 Sen:     RT1 ile /refresh → ✓ yeni RT2 aldın.        🗄️ RT1: revoked_at = 14:00
😈 Hırsız: RT1 ile /refresh → rotate() ② "RT1 zaten kullanılmış!" 🚨
                            → revokeAllActiveByUserId → 🗄️ RT2 de iptal
                            → 401
📱 Sen:     15 dk sonra RT2 ile /refresh → ② kullanılmış (iptal edildi) → 401 → login ekranı
```

Durum B: Önce hırsız kullanıyor

```
😈 Hırsız: RT1 ile /refresh → ✓ RT2 aldı (artık hırsızda)     🗄️ RT1: iptal
📱 Sen:     RT1 ile /refresh → ② "RT1 zaten kullanılmış!" 🚨
                            → bütün token'lar iptal (hırsızın RT2'si DAHİL)
                            → 401 → sen login ekranına düşersin
😈 Hırsız: 15 dk sonra RT2 ile /refresh → iptal edilmiş → 401 → DIŞARIDA
```

Sonuç

Kim önce kullanırsa kullansın, ikinci kullanımda sistem hırsızlığı fark ediyor ve **herkesi** dışarı atıyor. Sen şifrenle tekrar girersin. Hırsızın şifren olmadığı için giremez. Backend hangisinin gerçek kullanıcı olduğunu bilemez, bu yüzden en güvenli yol herkesi dışarı atmak.

Bunu Postman ile kendin deneyebilirsin: login ol, RT1 ile refresh yap (RT2 gelir), sonra **tekrar RT1** ile refresh yap. 401 gelmeli ve DB'de o kullanıcının bütün satırlarının `revoked_at` alanı **aynı anda** dolmalı.

Kod ve çalışan SQL

```
if (current.getRevokedAt() != null) {                                  // daha önce kullanılmış
    refreshTokenRepository.revokeAllActiveByUserId(user.getId(), now); // BÜTÜN oturumları kapat
    throw new InvalidRefreshTokenException();                          // 401
}

-- Çalışan SQL:
UPDATE refresh_tokens SET revoked_at = ? WHERE user_id = 3 AND revoked_at IS NULL
```

Neden sadece o isteği reddetmekle yetinmiyoruz?

Backend **kimin hırsız olduğunu bilmiyor.** Bildiği tek şey "bu token iki kez kullanıldı". Durum B'de ikinci getiren sensin. Sadece gelen isteği reddetseydik seni reddetmiş olurduk, hırsız da RT2 ile içeride kalırdı. Herkesi dışarı atınca gerçek sahibi şifresiyle geri geliyor, hırsız gelemiyor.

`revoked_at` sadece yenilemede dolmuyor

Ne oldu?	`revoked_at` doldu mu?	O token tekrar gelirse
Yenileme (rotation)	✅	🚨 Bütün oturumlar kapatılır
Çıkış (logout)	✅	🚨 Bütün oturumlar kapatılır (çıkış yapılmış bir token'ı normal kullanıcı asla tekrar göndermez)
Ban veya tekrar kullanım (`revokeAll`)	✅	🚨 (zaten hepsi kapalı)

Hırsızın o anda yakalanmadığı tek durum: `revoked_at` **boşken**, yani hırsız token'ı senden önce kullandığında. Ayrıntısı bölüm 16'da.

13. Senaryo: Kullanıcı banlandı

Veritabanında kullanıcının `active` alanını `false` yaptın.

- **Bir sonraki normal istek:** Filtredeki `CustomUserDetailsService` kullanıcıyı pasif görür (`isEnabled() = false`) ve bileklik takılmaz → 401.
- **Mobil yenilemeyi dener:** `rotate()` ④ numaralı kontrol kullanıcının pasif olduğunu görür, bütün token'larını iptal eder ve 401 döner → login ekranı.
- **Tekrar giriş denerse:** `AuthService.login` içindeki `!user.isActive()` kontrolü → 401.

Yani ban **anında** etki ediyor. Bu, filtrenin her istekte DB'ye bakmasının faydası.

14. Veritabanı tablosunu okumak

Kolon	Ne tutuyor?	Kim dolduruyor?
id	Satır numarası	Veritabanı (otomatik artan)
user_id	Token kime ait	`create()` → `setUser`
token_hash	Token'ın SHA-256 hash'i (64 karakter), kendisi değil	`create()` → `setTokenHash(hash(raw))`
created_at	Satırın DB'ye eklendiği an	Hibernate (`@CreationTimestamp`)
expires_at	Son geçerlilik = oluşturulma + 30 gün	`create()` → `now().plus(ttl)`
revoked_at	**NULL = aktif**, dolu = iptal edildiği an	`rotate()` , `revoke()` , `revokeAll`
updated_at	En son değiştirildiği an	Hibernate (`@UpdateTimestamp`)

Gerçek tablodan örnek

id	created_at	revoked_at	Ne oldu?
15	14:51:08	14:51:10	15 iptal edildikten **32 saniye** sonra 16 geldi → **çıkış + tekrar giriş**, yenileme değil
16	14:51:42	NULL	
18	14:54:48	14:56:16	Eski iptal edildi, hemen ardından yenisi doğdu → **sessiz yenileme (rotation)**
19	14:56:19	15:07:47	
20	15:07:48	NULL	

- **Hiç iptal edilmemiş eski satırlar** (yukarıda 16): Çıkış yapılmadan bırakılmış oturumlar (uygulama yeniden yüklendi, başka bir cihaz veya web). Zararsızlar, 30 gün sonra kendiliğinden geçersiz olurlar. Her cihaz ayrı bir satır.
- `updated_at` **:** Hiç değişmemiş satırlarda `created_at` ile aynı. Tek tek iptal edilen satırlarda (`rotate` , `revoke`) iptal anını alır. Ama toplu iptalde (`revokeAllActiveByUserId`) **değişmez**, çünkü toplu `UPDATE` sorgusu Hibernate'in entity mekanizmasını atlıyor.
- **Normalde birkaç milisaniye olan farklar bazen saniyelere çıkıyorsa:** `expires_at` Java'da (`now()`), `created_at` ise Hibernate'in kayıt yaptığı anda hesaplanıyor. Debugger'la bir breakpoint'te beklerken aradaki fark saniyelere çıkabilir. Zararsız.

15. Hash karşılaştırması: BCrypt ve SHA-256

Gelen refresh token neyle karşılaştırılıyor?

```
Login'de (create):    ham token ──SHA-256──► hash ──► tabloya YAZ   (ham hali telefona gider)
Refresh'te (rotate):  ham token ──SHA-256──► hash ──► tabloda ARA
                                                        ▲
                                  aynı token → her zaman aynı hash

SQL:  SELECT * FROM refresh_tokens WHERE token_hash = '3e8f453e...994111'
      → bulunan satır = iptal edilecek satır → setRevokedAt(now) → yeni satır
```

"Şifresi 123 olan iki kullanıcının hash'i farklı. Token'da nasıl aynı çıkıyor?"

Çünkü iki farklı yöntem kullanıyoruz:

	Şifre: BCrypt	Refresh token: SHA-256
Salt (rastgele ek değer)	Var, her seferinde yeni. Hash'in içinde saklanıyor.	Yok
Aynı girdi → aynı hash?	❌ Hayır	✅ Evet
Hız	Bilerek yavaş (~100 ms)	Çok hızlı
Karşılaştırma	① `findByUserName` ile kişiyi bul ② `matches()` o kişinin hash'indeki salt'ı okuyup aynı şekilde hesaplar	① `hash(token)` ② `findByTokenHash` ile doğrudan ara
Neden?	Şifreler zayıf (`123456`): yavaşlık ve salt deneme-yanılmayı ve hazır tabloları engeller	Token 256 bit rastgele: deneme-yanılma imkânsız. Refresh'te sadece token geldiği için **aranabilir** olmak zorunda.

Yanlış olanı seçseydik

Şifreyi SHA-256 ile hashlemek **güvensiz** olurdu: DB çalınınca zayıf şifreler saniyeler içinde kırılır. Refresh token'ı BCrypt ile hashlemek ise **çalışmazdı**: Her seferinde farklı hash çıktığı için `WHERE token_hash = ?` araması yapılamaz. Ayrıntılı karşılaştırma için `docs/sifre-ve-token-hashleme.pdf` .

16. Token çalınırsa: sistemin sınırları

Bir hırsız refresh token'ını ele geçirdi. Ne yapabilir?

- Refresh token ile **doğrudan** API'ye giremez (JWT değil, filtre onu çözemez).
- `/auth/refresh` ile yeni bir access token alırsa, o access token ile senin yapabildiğin her şeyi yapabilir: harcamaları görmek, eklemek, silmek.
- Şifreni öğrenemez. Oturum kapandıktan sonra tekrar giremez.

Kim önce kullanır?	Sonuç
Sen (en olası durum: uygulamayı kullanıyorsan token çok sık yenileniyor)	Hırsızın token'ı zaten iptal edilmiş. Kullanmaya çalıştığı anda yakalanır, hiçbir şey elde edemez.
Hırsız	Sen uygulamayı açana kadar içeride. Açtığın anda senin uygulaman eski token'ı gönderir, hırsızlık yakalanır ve herkes dışarı atılır.

Zayıf nokta: Refresh token süresi aktif bir hırsızı sınırlamaz

Rotation her yenilemede **yeni bir 30 gün** veriyor. Hırsız düzenli yeniledikçe süresi hiç dolmaz. Refresh token süresi 2 gün bile olsa, sen uygulamayı açmadığın sürece hırsız **süresiz** içeride kalabilir. Refresh token süresi sadece **kullanılmayan** çalıntı bir token'ı öldürür.

Sen ne yapıyorsun?	Hırsız ne kadar içeride kalır?
Uygulamayı her gün kullanıyorsun	Senin bir sonraki yenilemene kadar (dakikalar veya saatler)
Haftada bir açıyorsun	En fazla bir hafta
Hiç açmıyorsun	Süresiz (hırsız yenilemeye devam ettiği sürece)

Bu, rotation yönteminin bilinen bir sınırı. Kapatmanın yolu bir sonraki bölümdeki **mutlak oturum süresi**.

17. Mutlak oturum süresi (henüz yok)

	Hareketsizlik süresi (idle)	Mutlak süre (absolute)
Sorduğu soru	"En son ne zaman kullanıldı?"	"**İlk** ne zaman giriş yapıldı?"
Saat ne zaman sıfırlanıyor?	Her yenilemede	Hiçbir zaman
Projede	✅ 30 gün	❌ Yok (bilinçli olarak şimdilik eklenmedi)

```
Mutlak süre 90 gün olsaydı:
1 Ocak    login               → oturum başlangıcı: 1 Ocak
20 Ocak   refresh             → yeni token, başlangıç hâlâ 1 Ocak
...
2 Nisan   refresh             → "1 Ocak'ta başlamış, 90 gün geçti" → 401 → login
```

Hangi süre önce dolarsa oturum o zaman biter: 30 gün hiç kullanmazsan, veya her gün kullansan bile 90 gün geçince. Fark edilmemiş bir hırsız da en fazla mutlak süre kadar kalabilir.

Eklemek istenirse: Entity'ye `session_started_at` kolonu (login'de "şimdi", rotation'da eski token'dan kopyalanır), yaml'a `absolute-ttl: 90d`, `rotate()`'e bir kontrol, ve `expiresAt = min(şimdi + 30 gün, başlangıç + 90 gün)`. Mobilde değişiklik gerekmez. Dikkat: Tablo dolu olduğu için yeni kolon `NOT NULL` eklenemez (`User.userName`'deki sorunun aynısı).

18. Süreler: ne kadar olmalı?

Tek bir doğru sayı yok. Uygulamanın ne kadar hassas olduğuna ve kullanıcı deneyimine göre seçiliyor.

Uygulama türü	Access	Refresh (hareketsizlik)	Refresh (mutlak)
Sosyal medya, alışveriş (mobil)	15 dk	30–90 gün	90 gün – 1 yıl, veya hiç
Genel web uygulaması	15 dk	1–14 gün	30 gün
Kurumsal uygulama	5–15 dk	8–24 saat	1–7 gün
Bankacılık	5 dk	5–15 dk	Birkaç saat

- **Access token için 15 dakika iyi bir denge.** Çok kısa tutmanın da bedeli var: Her süre dolduğunda bir refresh isteği gidiyor ve tablo büyüyor.
- **Bankalar** farklı bir model kullanıyor: kısa oturum (dakikalar ve saatler), uygulamayı her açışta PIN veya biyometri, para transferi gibi hassas işlemlerde tekrar doğrulama (SMS, mobil onay) ve cihaz bağlama.
- **Referans standart:** OWASP ASVS, kritik sistemler için saatler ve dakikalar, düşük riskli sistemler için günler mertebesinde yeniden doğrulama öneriyor. (Kesin değerler için güncel sürüme bakılmalı.)

Bu proje için öneri

Harcama takip uygulaması finansal veri içeriyor ama para transferi yapmıyor. **Access 15 dakika, refresh 30 gün hareketsizlik, (ileride) 90 gün mutlak.** Daha sıkı olmak istenirse token sürelerini kısaltmak yerine **uygulama kilidi** (Face ID / PIN, `expo-local-authentication`) eklemek hem güvenlik hem kullanıcı deneyimi açısından daha iyi.

Şu anki ayar

Yaml'da `access-token-ttl: 30s` test için. Canlıda kesinlikle `15m` olmalı.

19. Hırsızın elini kolunu bağlamak: güvenlik katmanları

%100 güvenlik mümkün değil. Sunucu, aynı token'ı gönderen iki kişiden hangisinin gerçek olduğunu kendi başına bilemez. Telefona tam yetkili zararlı bir yazılım bulaşmışsa hiçbir token tasarımı bunu tamamen önleyemez. Ama katmanları üst üste koydukça hırsızın işi çok zorlaşır.

Katman	Önlem	Projede
1. Çalınmasını önle	**HTTPS** (yoksa aynı Wi-Fi'deki biri token'ları düz yazı görür)	❌ Şu an `http://`
	Şifreli depolama (SecureStore)	✅
	DB'de sadece hash	✅
	Token'ları loglamamak	✅
2. Çalınanı işe yaramaz yap	**DPoP** (RFC 9449): Token, telefonun güvenli çipindeki çıkarılamaz bir anahtara bağlanır, her istek bu anahtarla imzalanır. Token'ı çalmak yetmez. "Kart sadece senin parmak izinle çalışır."	❌ İleri seviye
3. Tespit et	Tekrar kullanım tespiti	✅
	Şüpheli değişiklikler, "yeni cihazdan giriş" bildirimi	❌
4. Zararı sınırla	Kısa access token	⚠️ `30s` → `15m`
	Mutlak oturum süresi	❌
	Uygulama kilidi (Face ID / PIN)	❌
5. Müdahale et	"Tüm cihazlardan çıkış"	⚠️ `revokeAll()` hazır, endpoint yok
	Şifre değişince tüm oturumları kapatmak, aktif oturum listesi	❌

Önerilen sıra

1. Canlıda **HTTPS**. Olmazsa diğer hiçbir şeyin anlamı yok.
2. **Access token 15 dakika.** Tek satır.
3. **Mutlak oturum süresi.** Aktif hırsız açığını kapatıyor.
4. **"Tüm cihazlardan çıkış".** `revokeAll` hazır.
5. **Uygulama kilidi.**
6. **DPoP.** Ancak gerçekten yüksek güvenlik gerekirse.

20. Sık sorulan "saf" sorular

❓ Her istekte refresh_tokens tablosuna veri ekleniyor mu?

Hayır. Sadece login'de, register'da ve **access token'ın süresi dolduktan sonraki ilk istekte** (refresh) ekleniyor. 15 dakikalık token ile, uygulamayı aralıksız kullanan biri için saatte en fazla 4 satır.

❓ Test sırasında neden bu kadar çok satır oluştu?

Çünkü `access-token-ttl: 30s` idi. 30 saniyede bir token ölüyordu ve neredeyse her ekran değişimi yenileme tetikliyordu.

❓ Hırsız refresh token ile harcamalarımı görebilir mi?

Doğrudan göremez. Refresh token JWT değil. `Authorization` başlığına koysa filtre onu çözemez ve 401 alır. Yapabileceği tek şey `/refresh` ile yeni access token almaya çalışmak. Orada da tekrar kullanım tespiti devreye giriyor.

❓ Veritabanı çalınırsa ne olur?

Hırsızın eline sadece hash'ler geçer. Hash'ten token hesaplanamaz, dolayısıyla `/refresh` isteği atamaz. Şifreler de BCrypt ile hash'li. Gizli anahtar (`JWT_SECRET`) ise DB'de değil, ortam değişkeninde.

❓ Logout'tan sonra access token neden hâlâ çalışabiliyor?

Çünkü JWT geri alınamaz. Logout sadece refresh token'ı iptal ediyor. Telefon access token'ı sildiği için normal kullanımda önemi yok. Ama kopyası çalınmışsa en fazla 15 dakika çalışır. Süreyi kısa tutmamızın sebebi tam olarak bu.

❓ Neden refresh token'ı her seferinde değiştiriyoruz (rotation)?

Değiştirmeseydik çalınan bir refresh token 30 gün boyunca fark edilmeden kullanılabilirdi. Değiştirdiğimiz için eski token'ın ikinci kullanımı hırsızlığı ele veriyor.

❓ Neden cookie kullanmıyoruz?

Cookie tarayıcıların özelliği. Mobil uygulamada token'ı kendimiz SecureStore'da saklayıp başlığa ekliyoruz. Bu yüzden SecurityConfig'te CSRF koruması kapalı: CSRF saldırısı, tarayıcının cookie'yi otomatik eklemesine dayanıyor ve bizde bu yok.

❓ Bir kullanıcının birden fazla satırı olması normal mi?

Evet. Her cihaz (telefon, tablet, web) ve çıkış yapılmadan bırakılmış her oturum ayrı bir satır. İptal edilmemiş olanlar 30 gün sonra kendiliğinden geçersiz olur.

❓ Tablo sonsuza kadar büyür mü?

Şu an evet, yavaşça büyür. İleride her gece süresi dolmuş satırları silen bir zamanlanmış iş (`@Scheduled`) eklenebilir. Süresi dolmuş satırları silmek güvenli, çünkü o token'lar zaten reddediliyor.

❓ Refresh token süresi 2 gün olsa, hırsız en fazla 2 gün mü kalır?

Hayır. Hırsız token'ı kullanmaya başladıysa her yenilemede yeni bir 2 gün alır. Sen uygulamayı açmadığın sürece süresiz kalabilir. Onu yakalayan şey senin login olman değil, **uygulamayı açman**: Uygulama eski token'ı gönderir ve tekrar kullanım tespiti devreye girer. Aktif bir hırsıza kesin sınır koyan şey mutlak oturum süresi (bölüm 17).

❓ Gönderdiğim refresh token'ın `revoked_at` alanı doluysa ne oluyor?

Backend "bu token daha önce kullanılmış, çalınmış" der ve o kullanıcının bütün aktif refresh token'larını iptal eder. Kimin hırsız olduğunu bilemediği için herkesi dışarı atar. Gerçek sahibi şifresiyle geri girer.

❓ Hırsızın elini kolunu tamamen bağlamak mümkün mü?

%100 değil. Ama katmanlar eklendikçe (HTTPS, kısa access token, mutlak süre, uygulama kilidi, DPoP) token'ı çalmak zorlaşıyor, çalınsa bile kullanmak zorlaşıyor, kullanılsa bile fark ediliyor ve zarar sınırlı kalıyor. Bkz. bölüm 19.

❓ "401" ve "403" farkı ne?

401 = "seni tanımıyorum" (token yok veya geçersiz). **403** = "seni tanıyorum ama bu işe yetkin yok". Mobil 401 görünce yenileme yapıyor. `SecurityConfig`'teki `HttpStatusEntryPoint` satırı kimliksiz isteklerde her zaman 401 dönülmesini sağlıyor.

21. Tek sayfalık özet ve yapılacaklar

Refresh token'ın mantığı, tek paragrafta

> Login olduğunda backend sana iki token verir: her istekte kullanılan, kısa ömürlü (15 dk) bir **access token** ve sadece yeni access token almaya yarayan, uzun ömürlü (30 gün) bir **refresh token**. Access token bir JWT'dir. Sunucu onu saklamaz, sadece imzasına ve süresine bakar. Bu yüzden hızlıdır ama geri alınamaz. Refresh token ise rastgele bir yazıdır ve sunucu onun **hash'ini** veritabanında saklar, bu sayede istendiği an iptal edilebilir. Access token'ın süresi dolduktan sonraki ilk istekte backend 401 döner. Mobil uygulama sen fark etmeden refresh token'ı `/auth/refresh` endpoint'ine gönderir. Backend token'ı hash'leyip tabloda arar ve dört şeyi kontrol eder: var mı, daha önce kullanılmış mı, süresi dolmuş mu, kullanıcı aktif mi. Her şey yolundaysa eski refresh token'ı iptal eder, yeni bir access token ve yeni bir refresh token verir. Mobil de 401 alan isteği yeni token'la tekrar gönderir. Refresh token'ın her seferinde yenilenmesine **rotation** denir: Çalınan token kısa sürede bayatlar, ve iptal edilmiş bir token tekrar gelirse backend bunun çalındığını anlayıp kullanıcının bütün oturumlarını kapatır. Çıkışta da refresh token iptal edilir. Böylece oturum gerçekten biter, kullanıcı da 30 gün boyunca uygulamayı hiç açmadığı sürece tekrar şifre girmek zorunda kalmaz.

Akışın doğru ifadesi

Login → access (15 dk) + refresh (30 gün). Access geçerliyken istekler normal çalışır, refresh tablosuna dokunulmaz. Süre dolduktan sonraki **ilk istekte** 401 gelir. Mobil **otomatik olarak** (ve aynı anda tek sefer) refresh yapar. Backend refresh token'ı 4 açıdan kontrol eder, eskisini **iptal eder** (silmez), yeni iki token verir. Mobil 401 alan isteği **tekrar gönderir**.



Soru	Cevap	Nerede?
Access token'ı kim üretir?	JwtService	`generateToken`
Access token'ın süresine kim bakar?	JJWT kütüphanesi (`exp` alanı)	`JwtService.extractUserName`
401'i kim döner?	SecurityConfig	`authenticated()` + `HttpStatusEntryPoint`
401'i kim yakalar?	Mobil apiClient	`request()` ② adımı
Refresh token'ı kim üretir?	RefreshTokenService	`create`
Refresh token'ın geçerliliğine kim bakar?	RefreshTokenService (senin kodun)	`rotate` ①–④
Tabloya satır ne zaman eklenir?	Login, register, refresh	`create`
Satır ne zaman iptal edilir?	Refresh, logout, hırsızlık, ban	`rotate` , `revoke`
Token'lar telefonda nerede?	SecureStore (web'de localStorage)	`tokenStorage`
Aynı anda tek yenilemeyi kim sağlar?	apiClient	`refreshPromise`
Hangi ekranın gösterileceğine kim karar verir?	_layout (Stack.Protected)	`token !== null`

Durum ve yapılacaklar

Konu	Durum
Refresh token, rotation, tekrar kullanım tespiti, gerçek çıkış	✅ Çalışıyor (DB kayıtlarıyla doğrulandı)
`access-token-ttl` → `15m`	⚠️ Şu an `30s` (test)
Tekrar kullanım tespitini Postman ile canlı denemek	⏳ Yapılmadı
Canlıda HTTPS	⏳ Canlıya çıkarken
Mutlak oturum süresi	⏸️ Şimdilik ertelendi
"Tüm cihazlardan çıkış" (endpoint + buton)	⏳ `revokeAll()` hazır
Süresi dolmuş satırları temizleyen `@Scheduled` iş	⏳ İsteğe bağlı
`JwtAuthenticationFilter` 'daki "24 saat" yorumunu güncellemek	⏳ Küçük düzeltme

Tek cümleyle

Giriş yapınca kısa ömürlü bir **oda kartı** (access token) ve uzun ömürlü bir **konaklama belgesi** (refresh token) alırsın. Kart her kapıda okunur. Kart ölünce telefon sen fark etmeden belgeyle yenisini alır ve resepsiyon eski belgeyi yırtar. Çıkışta belge iptal edilir. Yırtılmış bir belgeyle gelen olursa resepsiyon hırsızı anlar ve herkesi dışarı çıkarır.